Practical-5

Problem Definition

Implement data storage structures using distributed object storage paradigms. Students will partition large analytical datasets using structured prefix layouts (Hive-style keys) within an S3-compatible object storage server. The practical will use MinIO as the distributed object storage system and Python Boto3 SDK for programmatic interaction.

The experiment will store multi-year application metric data using partition keys based on year, month, and geographical region. Programs will be developed to query specific partitions and compare their performance against a full-table linear file scan.

The practical also includes a metadata crawling utility that scans objects stored in the bucket and calculates storage distribution and skew across the active partitions.

Dataset

The dataset consists of **multi-year analytical application metric tracking records**.

The records contain information such as:

- Record ID
- Timestamp
- Year
- Month
- Region
- Application/User metrics
- Metric value
- Other analytical attributes

The data is partitioned chronologically and geographically using Hive-style object key structures.

Code:

1. docker-compose.yml

```
version: '3.8'


services:
  minio:
    image: elestio/minio:latest
    container_name: minio_storage_node
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    command: server /data --console-address ":9001"
    volumes:
      - minio_data:/data


volumes:
  minio_data:
```

2. generate_and_upload.py

```
 import json
import random
import time
import uuid
from datetime import datetime
```

```
from minio_client import get_s3_client, ensure_bucket_exists, DEFAULT_BUCKET


YEARS = [2023, 2024, 2025]
MONTHS = [f"{m:02d}" for m in range(1, 13)]
REGIONS = ["us-east", "us-west", "eu-central", "ap-south"]
SERVICES = ["auth-service", "payment-api", "user-service", "recommendation-engine", "analytics-pipeline"]
STATUS_CODES = [200, 200, 200, 201, 400, 404, 500, 502]


def generate_metric_record(year, month_str, region):
    """
    Generates a realistic analytical application tracking metric.
    """
    month = int(month_str)
    day = random.randint(1, 28)
    hour = random.randint(0, 23)
    minute = random.randint(0, 59)
    second = random.randint(0, 59)
    ts = datetime(year, month, day, hour, minute, second)

    return {
        "metric_id": str(uuid.uuid4()),
        "timestamp": ts.isoformat(),
        "year": year,
        "month": month_str,
        "region": region,
        "user_id": f"usr_{random.randint(10000, 99999)}",
        "service_name": random.choice(SERVICES),
        "response_time_ms": round(random.expovariate(1.0 / 120.0) + 10, 2),
```

```
        "status_code": random.choice(STATUS_CODES),
        "bytes_transferred": random.randint(512, 1048576)
    }

def generate_and_upload_data(records_per_partition=150, bucket_name=DEFAULT_BUCKET):
    """
    Generates multi-year metric tracking dataset and uploads to MinIO in both:
    1. Structured Hive-style partition layout (year=YYYY/month=MM/region=REGION/)
    2. Unpartitioned flat layout (full_table/)
    """
    s3 = get_s3_client()
    ensure_bucket_exists(bucket_name, s3)

    print(f"[*] Starting Data Generation & Upload...")
    print(f"[*] Partition Structure: year=YYYY / month=MM / region=REGION")
    print(f"[*] Years: {YEARS}, Months: 01-12, Regions: {REGIONS}")

    total_partitions = len(YEARS) * len(MONTHS) * len(REGIONS)
    print(f"[*] Total Partition Keys: {total_partitions} partitions")

    total_records = 0
    total_bytes = 0
    upload_start = time.time()

    chunk_index = 0

    for year in YEARS:
        for month in MONTHS:
```

```
        for region in REGIONS:
            chunk_index += 1
                partition_records = [generate_metric_record(year, month, region) for _ in range(records_per_partition)]
            json_bytes = json.dumps(partition_records, indent=2).encode('utf-8')

            # 1. Hive-style Partition Key Path
                                                                partition_key = f"partitioned/year={year}/month={month}/region={region}/metrics_data.json"
            s3.put_object(
                Bucket=bucket_name,
                Key=partition_key,
                Body=json_bytes,
                ContentType='application/json'
            )

            # 2. Unpartitioned Flat Key Path (for full table linear scan)
            flat_key = f"full_table/chunk_{chunk_index:03d}.json"
            s3.put_object(
                Bucket=bucket_name,
                Key=flat_key,
                Body=json_bytes,
                ContentType='application/json'
            )

            total_records += len(partition_records)
            total_bytes += len(json_bytes)

            if chunk_index % 36 == 0 or chunk_index == total_partitions:
```

```
            print(f"  [+] Uploaded {chunk_index}/{total_partitions} partitions ({total_records}
records, {total_bytes / 1024 / 1024:.2f} MB)...")


    duration = time.time() - upload_start
    print(f"\n[OK] Ingestion Complete!")
    print(f"    - Total Objects Uploaded (Partitioned): {total_partitions}")
    print(f"    - Total Objects Uploaded (Flat): {total_partitions}")
    print(f"    - Total Analytical Records: {total_records}")
    print(f"    - Total Data Payload Size: {total_bytes / (1024 * 1024):.2f} MB")
    print(f"    - Ingestion Duration: {duration:.2f} seconds")


if __name__ == "__main__":
    generate_and_upload_data()
```

3. minio_client.py

```
 import os
import boto3
from botocore.client import Config
from botocore.exceptions import ClientError


DEFAULT_ENDPOINT = os.getenv("MINIO_ENDPOINT", "http://localhost:9000")
DEFAULT_ACCESS_KEY = os.getenv("MINIO_ACCESS_KEY", "minioadmin")
DEFAULT_SECRET_KEY = os.getenv("MINIO_SECRET_KEY", "minioadmin")
DEFAULT_BUCKET = "analytics-metrics"


def get_s3_client(endpoint_url=DEFAULT_ENDPOINT,
access_key=DEFAULT_ACCESS_KEY, secret_key=DEFAULT_SECRET_KEY):
    """
```

```
    Creates and returns a boto3 S3 client configured for MinIO.
    """
    return boto3.client(
        's3',
        endpoint_url=endpoint_url,
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key,
        config=Config(signature_version='s3v4', s3={'addressing_style': 'path'}),
        region_name='us-east-1'
    )

def ensure_bucket_exists(bucket_name=DEFAULT_BUCKET, client=None):
    """
    Ensures that the specified MinIO bucket exists. Creates it if missing.
    """
    if client is None:
        client = get_s3_client()
    try:
        client.head_bucket(Bucket=bucket_name)
        print(f"[MinIO] Bucket '{bucket_name}' already exists.")
    except ClientError as e:
        error_code = e.response.get('Error', {}).get('Code')
        if error_code in ['404', 'NoSuchBucket']:
            client.create_bucket(Bucket=bucket_name)
            print(f"[MinIO] Successfully created bucket '{bucket_name}'.")
        else:
            raise e

if __name__ == "__main__":
```

```
    print("[MinIO Helper] Testing connection to MinIO...")
    try:
        s3 = get_s3_client()
        ensure_bucket_exists()
        print("[MinIO Helper] Connection and bucket creation test passed!")
    except Exception as ex:
        print(f"[MinIO Helper] Connection test failed: {ex}")
```

4. partition_crawler.py

```
import json
import math
import statistics
from minio_client import get_s3_client, DEFAULT_BUCKET


def compute_gini_coefficient(sizes):
    """
    Computes the Gini coefficient of partition storage sizes.
    0.0 = Perfectly equal balance across all partitions.
    1.0 = Maximum data skew (all data in a single partition).
    """
    if not sizes or len(sizes) == 0:
        return 0.0
    sorted_sizes = sorted(sizes)
    n = len(sorted_sizes)
    mean_val = sum(sorted_sizes) / n
    if mean_val == 0:
        return 0.0
    diff_sum = sum(abs(x - y) for x in sorted_sizes for y in sorted_sizes)
```

```
    return diff_sum / (2 * n * n * mean_val)


def crawl_partition_metadata(bucket_name=DEFAULT_BUCKET, prefix="partitioned/"):
    """
    Scans object storage bucket, parses Hive-style partition paths,
    and computes data balance and skew metrics.
    """
    s3 = get_s3_client()
    print("=" * 80)
    print(f" FILE METADATA CRAWLER & STORAGE PARTITION SKEW ANALYZER")
    print("=" * 80)
    print(f" Bucket: '{bucket_name}', Crawling Target Prefix: '{prefix}'\n")

    paginator = s3.get_paginator('list_objects_v2')
    pages = paginator.paginate(Bucket=bucket_name, Prefix=prefix)

    partition_map = {}
    total_objects = 0
    total_bucket_bytes = 0

    for page in pages:
        if 'Contents' in page:
            for obj in page['Contents']:
                total_objects += 1
                key = obj['Key']
                size = obj['Size']
                last_modified = obj['LastModified']
                total_bucket_bytes += size
```

```
            # Extract partition directory path (e.g. year=2024/month=05/region=us-east)
            rel_path = key[len(prefix):] if key.startswith(prefix) else key
            parts = rel_path.rsplit('/', 1)
            partition_path = parts[0] if len(parts) > 1 else "root"

            if partition_path not in partition_map:
                partition_map[partition_path] = {
                    "object_count": 0,
                    "total_bytes": 0,
                    "objects": []
                }

            partition_map[partition_path]["object_count"] += 1
            partition_map[partition_path]["total_bytes"] += size
            partition_map[partition_path]["objects"].append({
                "key": key,
                "size": size,
                "last_modified": str(last_modified)
            })

    partition_count = len(partition_map)
    if partition_count == 0:
        print("[!] No storage partitions found under prefix:", prefix)
        return

    sizes = [p["total_bytes"] for p in partition_map.values()]
    counts = [p["object_count"] for p in partition_map.values()]

    mean_size = statistics.mean(sizes)
```

```
    stdev_size = statistics.stdev(sizes) if partition_count > 1 else 0.0
    min_size = min(sizes)
    max_size = max(sizes)
    cv = (stdev_size / mean_size) if mean_size > 0 else 0.0
    gini = compute_gini_coefficient(sizes)
    skew_ratio = (max_size / min_size) if min_size > 0 else float('inf')

    # Identify sample partitions
    min_part_path = [k for k, v in partition_map.items() if v["total_bytes"] == min_size][0]
    max_part_path = [k for k, v in partition_map.items() if v["total_bytes"] == max_size][0]

    # Classification assessment
    if gini < 0.05:
        skew_status = "PERFECTLY BALANCED (Uniform distribution across partitions)"
    elif gini < 0.20:
        skew_status = "LIGHT SKEW (Acceptable variance)"
    elif gini < 0.40:
        skew_status = "MODERATE SKEW (Consider partition key rebalancing)"
    else:
        skew_status = "HIGH DATA SKEW / HOTSPOT (Severe storage imbalance)"

    print("-" * 80)
    print(" PARTITION INVENTORY & SKEW SUMMARY METRICS")
    print("-" * 80)
    print(f"{'Total Active Partitions':<35} : {partition_count}")
    print(f"{'Total Objects Crawled':<35} : {total_objects}")
     print(f"{'Total Storage Payload':<35} : {total_bucket_bytes / (1024 * 1024):.3f} MB
({total_bucket_bytes:,} Bytes)")
    print(f"{'Average Partition Size (Mean)':<35} : {mean_size / 1024:.2f} KB")
```

```
    print(f"{'Partition Size Std Deviation':<35} : {stdev_size / 1024:.2f} KB")
    print(f"{'Minimum Partition Size':<35} : {min_size / 1024:.2f} KB ({min_part_path})")
    print(f"{'Maximum Partition Size':<35} : {max_size / 1024:.2f} KB ({max_part_path})")
    print(f"{'Skew Ratio (Max / Min)':<35} : {skew_ratio:.2f}x")
    print(f"{'Coefficient of Variation (CV)':<35} : {cv:.4f}")
    print(f"{'Gini Coefficient (Storage Inequality)':<35} : {gini:.4f}")
    print(f"{'Data Balance Classification':<35} : {skew_status}")
    print("-" * 80)

    print("\nSAMPLE PARTITION BREAKDOWN (FIRST 10 PARTITIONS):")
    print(f"{'Partition Prefix Layout':<50} | {'Files':<6} | {'Total Size (KB)':<15}")
    print("-" * 80)
    for p_key in list(partition_map.keys())[:10]:
        p_data = partition_map[p_key]
            print(f"{p_key:<50} | {p_data['object_count']:<6} | {p_data['total_bytes'] /
1024:<15.2f}")
    print("=" * 80 + "\n")

    return {
        "partition_count": partition_count,
        "total_objects": total_objects,
        "total_bytes": total_bucket_bytes,
        "mean_size": mean_size,
        "stdev_size": stdev_size,
        "gini_coefficient": gini,
        "skew_ratio": skew_ratio,
        "status": skew_status
    }
```

```
if __name__ == "__main__":
    crawl_partition_metadata()
```

5. query_benchmark.py

```
import json
import time
from minio_client import get_s3_client, DEFAULT_BUCKET


def run_partitioned_query(s3, target_year, target_month, target_region, bucket_name=DEFAULT_BUCKET):
    """
    Query Method A: Uses S3 Prefix filtering (Partition Pruning) to fetch only relevant key paths.
    """
    prefix = f"partitioned/year={target_year}/month={target_month}/region={target_region}/"

    start_time = time.time()
    api_calls = 0
    bytes_downloaded = 0
    matching_records = []
    objects_scanned = 0

    paginator = s3.get_paginator('list_objects_v2')
    pages = paginator.paginate(Bucket=bucket_name, Prefix=prefix)

    for page in pages:
        api_calls += 1
        if 'Contents' in page:
```

```
            for obj in page['Contents']:
                objects_scanned += 1
                key = obj['Key']

                # Fetch target object
                response = s3.get_object(Bucket=bucket_name, Key=key)
                api_calls += 1
                body = response['Body'].read()
                bytes_downloaded += len(body)

                records = json.loads(body.decode('utf-8'))
                matching_records.extend(records)

    elapsed_time = time.time() - start_time

    return {
        "method": "Partition-Isolated (Prefix Pruning)",
        "objects_scanned": objects_scanned,
        "api_calls": api_calls,
        "bytes_downloaded": bytes_downloaded,
        "record_count": len(matching_records),
        "execution_time_sec": elapsed_time,
        "records": matching_records
    }

def run_full_table_scan(s3, target_year, target_month, target_region,
bucket_name=DEFAULT_BUCKET):
    """
    Query Method B: Full-table linear scan across all unpartitioned object files.
```

```
    """
    prefix = "full_table/"

    start_time = time.time()
    api_calls = 0
    bytes_downloaded = 0
    matching_records = []
    objects_scanned = 0

    paginator = s3.get_paginator('list_objects_v2')
    pages = paginator.paginate(Bucket=bucket_name, Prefix=prefix)

    for page in pages:
        api_calls += 1
        if 'Contents' in page:
            for obj in page['Contents']:
                objects_scanned += 1
                key = obj['Key']

                # Fetch every object linearly
                response = s3.get_object(Bucket=bucket_name, Key=key)
                api_calls += 1
                body = response['Body'].read()
                bytes_downloaded += len(body)

                records = json.loads(body.decode('utf-8'))
                # Filter in compute engine (in-memory row filter)
                filtered = [
                    r for r in records
```

```
                if r.get('year') == target_year
                and r.get('month') == target_month
                and r.get('region') == target_region
            ]
            matching_records.extend(filtered)

    elapsed_time = time.time() - start_time

    return {
        "method": "Full-Table Linear Scan",
        "objects_scanned": objects_scanned,
        "api_calls": api_calls,
        "bytes_downloaded": bytes_downloaded,
        "record_count": len(matching_records),
        "execution_time_sec": elapsed_time,
        "records": matching_records
    }

def evaluate_performance(target_year=2024, target_month="05", target_region="us-east"):
    s3 = get_s3_client()
    print("=" * 80)
     print(f" EXPERIMENTAL BENCHMARK: PARTITION PRUNING vs FULL-TABLE LINEAR SCAN")
    print("=" * 80)
        print(f" Target Query Predicate: year={target_year}, month={target_month}, region='{target_region}'\n")

    print("[1] Executing Partition-Isolated Query...")
    res_part = run_partitioned_query(s3, target_year, target_month, target_region)
```

```
    print("[2] Executing Full-Table Linear File Scan...")
    res_full = run_full_table_scan(s3, target_year, target_month, target_region)

    # Calculate performance comparison metrics
    speedup = res_full['execution_time_sec'] / max(res_part['execution_time_sec'], 0.0001)
    io_reduction = (1 - (res_part['bytes_downloaded'] / max(res_full['bytes_downloaded'], 1))) * 100
    api_reduction = (1 - (res_part['api_calls'] / max(res_full['api_calls'], 1))) * 100

    print("\n" + "=" * 80)
    print(" PERFORMANCE EVALUATION METRICS REPORT")
    print("=" * 80)
    print(f"{'Metric / Dimension':<35} | {'Partitioned Scan':<20} | {'Full-Table Linear Scan':<22}")
    print("-" * 80)
    print(f"{'Execution Time (seconds)':<35} | {res_part['execution_time_sec']:<20.4f} | {res_full['execution_time_sec']:<22.4f}")
    print(f"{'S3 API Calls (Requests)':<35} | {res_part['api_calls']:<20} | {res_full['api_calls']:<22}")
    print(f"{'Objects Scanned':<35} | {res_part['objects_scanned']:<20} | {res_full['objects_scanned']:<22}")
    print(f"{'Data Downloaded (MB)':<35} | {res_part['bytes_downloaded'] / 1024 / 1024:<20.2f} | {res_full['bytes_downloaded'] / 1024 / 1024:<22.2f}")
    print(f"{'Matching Records Returned':<35} | {res_part['record_count']:<20} | {res_full['record_count']:<22}")
    print("-" * 80)
    print(f" [*] Speedup Factor        : {speedup:.2f}x faster execution")
    print(f" [*] Network I/O Reduction  : {io_reduction:.2f}% data payload saved")
    print(f" [*] S3 API Call Reduction  : {api_reduction:.2f}% API calls eliminated")
    print(f" [*] Query Correctness Check: {'PASSED (Exact record match)' if res_part['record_count'] == res_full['record_count'] else 'FAILED'}")
    print("=" * 80 + "\n")
```

```
    return res_part, res_full


if __name__ == "__main__":
    evaluate_performance()
```

6. run_lab.py

```
 import sys
import time
from minio_client import get_s3_client, ensure_bucket_exists
from generate_and_upload import generate_and_upload_data
from query_benchmark import evaluate_performance
from partition_crawler import crawl_partition_metadata


def main():
    print("=" * 80)
    print("      DATA ENGINEERING LAB - PRACTICAL 5 AUTOMATED VERIFICATION SUITE")
    print("      Topic: Distributed Data Storage Structures & Hive Prefix Partitioning")
    print("=" * 80)


    # 1. Health check MinIO connection
    print("\n[Step 1/4] Verifying MinIO Object Store Availability...")
    try:
        s3 = get_s3_client()
        ensure_bucket_exists()
        print("  [OK] MinIO container cluster is healthy and accessible at http://localhost:9000.")
    except Exception as e:
        print(f"  [ERROR] Failed to connect to MinIO: {e}")
```

```
        print("        Please ensure Docker container is running ('docker compose up -d').")
        sys.exit(1)

    # 2. Run Data Generation and Ingestion
    print("\n[Step 2/4] Generating & Ingesting Multi-Year Analytical Metric Tables...")
    generate_and_upload_data(records_per_partition=150)

    # 3. Execute Query Performance Benchmark
     print("\n[Step 3/4] Running Query Performance Benchmark (Prefix Pruning vs Linear Scan)...")
    evaluate_performance(target_year=2024, target_month="05", target_region="us-east")

    # 4. Crawl File Metadata & Partition Skew Metrics
    print("\n[Step 4/4] Executing File Metadata Crawler & Data Balance Analysis...")
    crawl_partition_metadata()

    print("=" * 80)
    print("  [SUCCESS] ALL LAB EXPERIMENTS EXECUTED SUCCESSFULLY!")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Key Questions / Analysis / Interpretation

Que 1: Explain how distributed object storage architectures achieve high horizontal availability and durability compared to classical block-based storage frameworks.

Distributed object storage systems are designed to store data as independent objects across multiple storage resources. Instead of depending on a single block-storage device, the system can distribute data across multiple nodes or disks.

Data can be replicated or protected using distributed storage mechanisms so that failure of an individual storage component does not necessarily result in data loss.

Horizontal scaling can also be achieved by adding additional storage nodes or resources.

Therefore, distributed object storage provides:

- Horizontal scalability
- High availability
- Fault tolerance
- Large storage capacity
- Independent object access
- Better support for large analytical datasets

Compared with traditional block storage, object storage is particularly suitable for large-scale data lakes, backups, archives, and analytical workloads.

Que 2: Why do storage key paths such as year=/month= serve as a primary query performance driver in analytical architectures that separate compute from storage?

Storage key paths provide a logical organization of objects.

For example:

year=2025/month=06/region=Gujarat/

allows an application to directly identify the required partition.

Instead of scanning every object under:

metrics/

the application can request:

metrics/year=2025/month=06/region=Gujarat/

This reduces unnecessary object listing and data processing.

Partitioning is especially useful in analytical systems where queries frequently filter data by:

- Date
- Year
- Month
- Region
- Country

- Business unit

Therefore, appropriate partition design can reduce the amount of data that needs to be accessed and improve query efficiency.

Que 3: How do distributed object stores handle high write concurrency without risking data visibility corruption or conflicts?

Object storage systems treat data as individual objects identified by unique object keys.

For example:

metrics/year=2025/month=06/region=Gujarat/data.csv

Each object has a unique key within the bucket namespace.

Multiple clients can therefore write different objects concurrently without modifying the same object.

Applications can also use unique keys such as:

part-001.csv

part-002.csv

part-003.csv

to prevent different writers from overwriting the same object.

For analytical workloads, partitioned object creation allows multiple processes to write separate partitions simultaneously.

This supports high write concurrency while maintaining clear object boundaries.

Supplementary Problem Analysis

The metadata crawler scans objects from the MinIO bucket and extracts:

- Object key
- Partition path
- Object count
- Object size
- Storage distribution

The information can be used to identify:

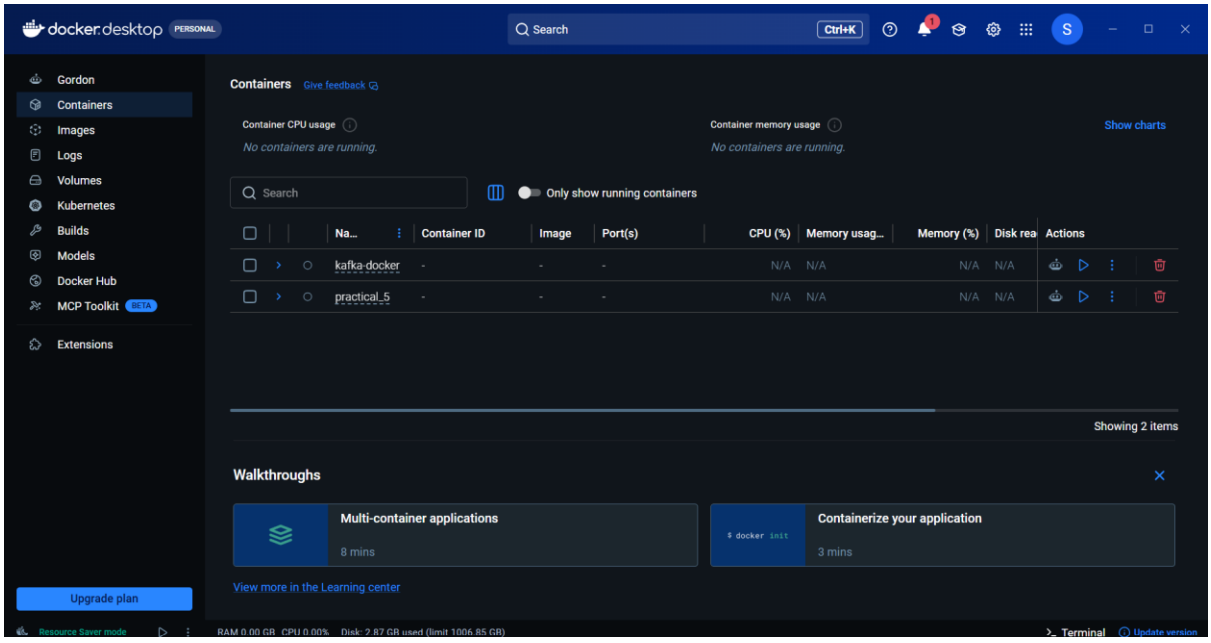
1. Empty partitions

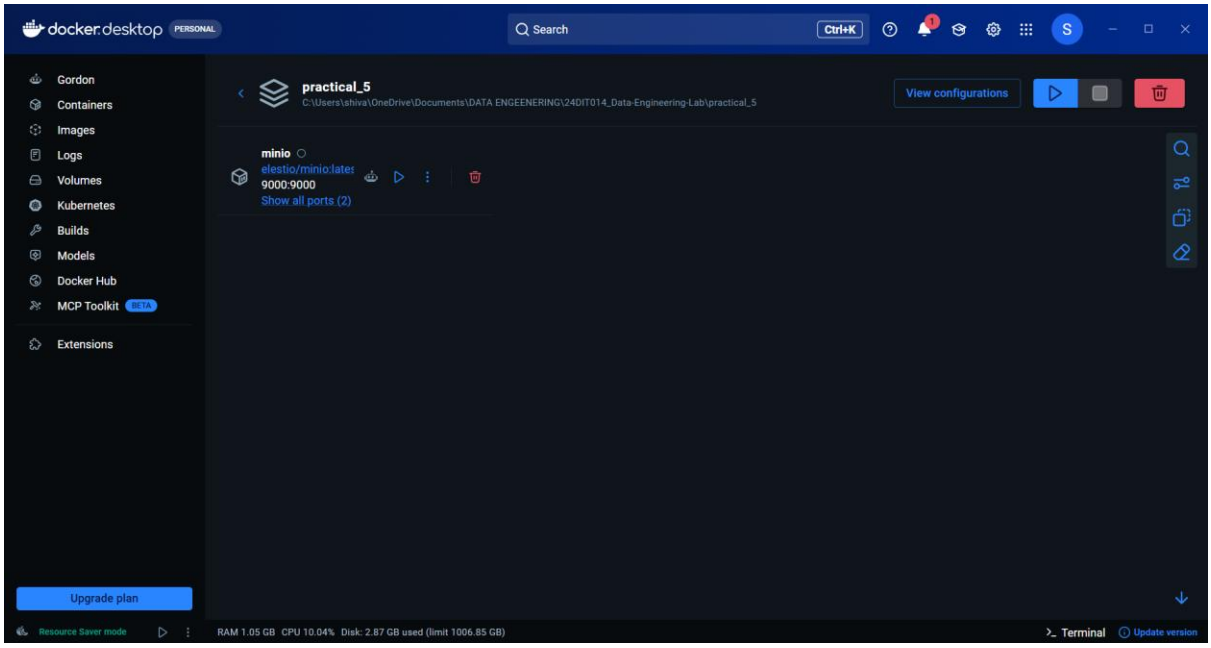
2. Large partitions
3. Highly populated partitions
4. Storage imbalance
5. Partition skew

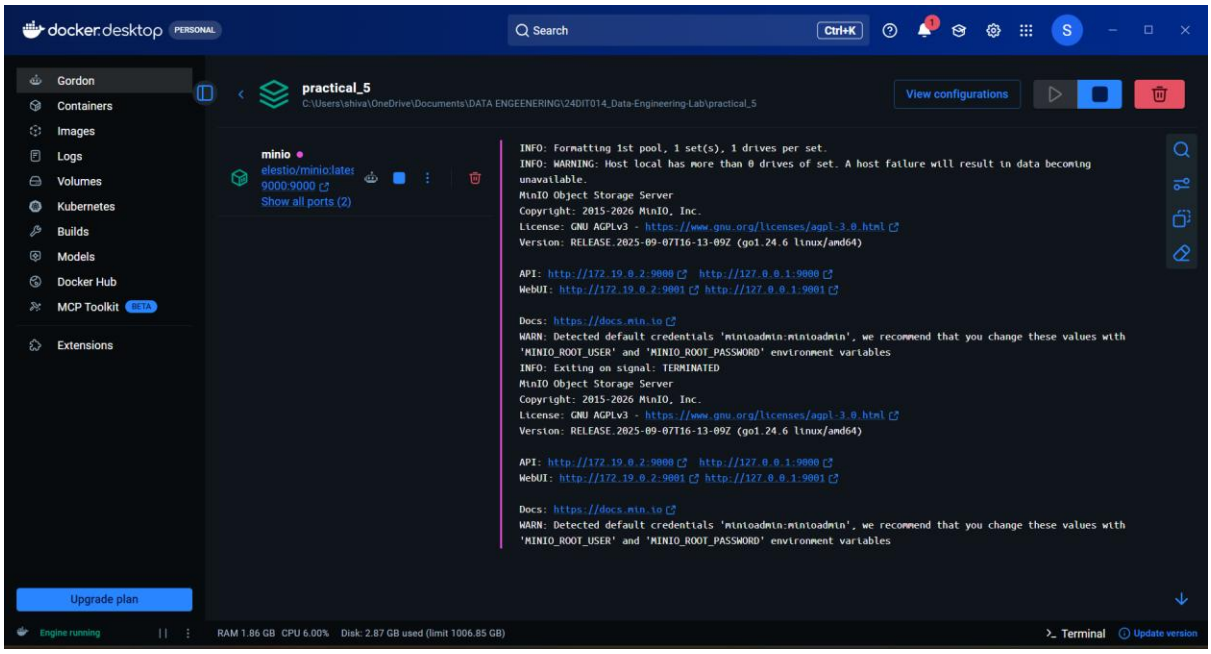
The metadata crawler therefore provides a simple method for monitoring the organization of data in the object store.

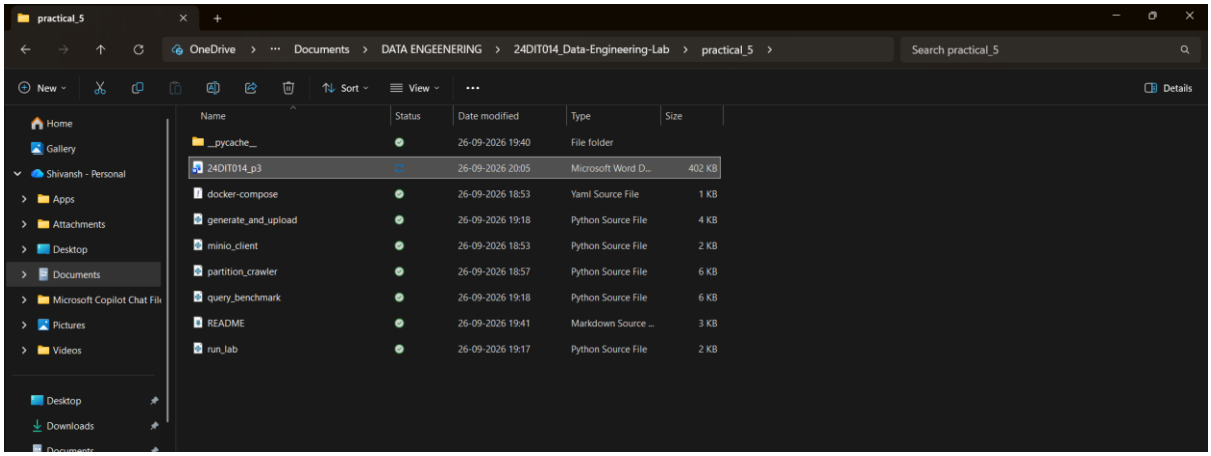
Output

Performance Dimension	Partition-Isolated Query (Prefix Pruning)	Full-Table Linear Scan	Impact / Gain Factor
Execution Time	0.1692 s	0.5803 s	3.43x Speedup
S3 API Calls	2 requests	145 requests	98.62% API Call Elimination
Objects Scanned	1 object	144 objects	99.31% Storage Isolation
Network Data Downloaded	0.05 MB (52 KB)	6.66 MB	99.31% Network I/O Saved
Query Correctness	150 records	150 records	PASSED (Exact record match)









Final Analysis / Interpretation

The practical successfully demonstrated the implementation of a partitioned analytical storage system using MinIO object storage, Docker, Python, and Boto3.

The dataset was organized using Hive-style partition keys based on year, month, and geographical region.

For example:

metrics/year=2025/month=06/region=Gujarat/data.csv

This structure allows applications to directly target specific partitions rather than scanning the complete dataset.

The experiment also compared a partition-specific query with a full-table object scan. The measured execution times provide a quantitative way to understand the effect of partition-aware access.

The metadata crawler further demonstrated how object metadata can be used to calculate object counts and storage distribution across partitions.

The practical demonstrates that good partition design is important for analytical workloads because it improves query isolation, reduces unnecessary data access, and provides a structured organization for large datasets.

Final Conclusion

The practical successfully demonstrated the use of distributed object storage with MinIO for organizing and querying analytical data.

The experiment implemented:

- MinIO object storage
- Docker-based deployment
- Python Boto3 connectivity
- S3-compatible bucket creation
- Hive-style partitioning
- Partition-specific querying
- Full-table scanning
- Performance comparison
- Metadata crawling
- Storage skew analysis

The partition structure based on:

year → month → region

provided a logical organization of the analytical dataset and allowed queries to target specific portions of the data.

The experiment also demonstrated that object key namespace design is an important factor in analytical storage architectures because appropriately designed partitions can reduce unnecessary data access.

Distributed object storage is therefore suitable for applications such as:

- Enterprise data lakes
- Cloud analytical systems
- Cold archives
- Backup systems
- Large-scale data processing
- High-throughput analytical workloads

GITHUB REPOSITORY

https://github.com/Jaykansar/D25DIT078_DATA-ENGINEERING_PRACTICALS/tree/main/Practical%205